

Guía Completa de GitHub para Principiantes: Conceptos y Comandos Esenciales

El control de versiones es una herramienta fundamental en el desarrollo de software moderno. Esta guía está diseñada para introducir a los principiantes en el ecosistema de Git y GitHub, proporcionando una base sólida desde la comprensión de los conceptos básicos hasta la ejecución de los primeros comandos.

1. ¿Qué es Git y qué es GitHub?

Es común confundir Git con GitHub, pero son tecnologías distintas que trabajan en conjunto:

- **Git:** Es un sistema de control de versiones de código abierto que se ejecuta localmente en tu computadora. Permite registrar los cambios realizados en los archivos a lo largo del tiempo.
- **GitHub:** Es una plataforma en la nube que aloja repositorios de Git. Facilita el trabajo colaborativo, el respaldo de código y la gestión de proyectos.

2. ¿Qué es un Repositorio?

Un repositorio (o "repo") es el espacio virtual o carpeta donde se almacenan todos los archivos de un proyecto, junto con el historial completo de sus modificaciones. Existen dos tipos esenciales:

- **Repositorio Local:** Se encuentra dentro de tu computadora personal.
- **Repositorio Remoto:** Se aloja en un servidor de internet, como GitHub, permitiendo el acceso a otros colaboradores.

3. Cómo Crear tu Propio Repositorio

Para iniciar un nuevo proyecto en GitHub, se deben seguir los siguientes pasos desde la interfaz web:

1. Inicia sesión en tu cuenta de GitHub.
2. En la esquina superior derecha, haz clic en el botón "+" y selecciona **"New repository"**.
3. Asigna un nombre claro y descriptivo a tu repositorio.
4. Elige la visibilidad del proyecto: **Public** (cualquiera puede verlo) o **Private** (solo tú y

quienes elijas pueden acceder).

- Opcionalmente, selecciona la casilla para agregar un archivo *README.md*, el cual sirve para describir el proyecto.
- Haz clic en el botón **"Create repository"**.

4. El Ciclo de Trabajo en Git: Áreas Principales

Antes de guardar cambios, es crucial comprender cómo Git clasifica los archivos en tu entorno local. La siguiente tabla detalla las tres áreas principales de trabajo:

Área de Trabajo	Descripción	Estado de los Archivos
Working Directory	El directorio local en tu computadora donde modificas los archivos directamente.	Modificados / Sin rastrear
Staging Area	Una zona de preparación donde se seleccionan los cambios que se incluirán en el próximo guardado.	Preparados (Staged)
Local Repository	La base de datos permanente de Git donde se guardan las versiones confirmadas de manera segura.	Confirmados (Committed)

5. ¿Qué es un Commit y Cómo Hacerlo?

Un **commit** es una captura o "instantánea" del estado de tus archivos en un momento específico. Equivale a guardar una partida en un videojuego o crear un punto de restauración en un documento de texto. Los commits permiten estructurar un historial claro y auditable, algo fundamental para el seguimiento y la evaluación precisa de cualquier avance.

Para realizar un commit desde la terminal de comandos, sigue esta secuencia:

Paso 1: Inicializar el repositorio (solo la primera vez en el directorio del proyecto)

```
git init
```

Paso 2: Vincular el repositorio local con el remoto en GitHub (solo la primera vez)

```
git remote add origin https://github.com/tu-usuario/tu-repositorio.git
```

Paso 3: Añadir los archivos al Staging Area

Puedes añadir un archivo específico o todos los archivos modificados utilizando un punto:

```
git add .
```

Paso 4: Confirmar los cambios con un Commit

Cada commit debe ir acompañado de un mensaje descriptivo que explique brevemente la modificación introducida:

```
git commit -m "Estructura inicial del proyecto y configuracion de archivos basicos"
```

6. Sincronización con el Repositorio Remoto

Una vez que los cambios están confirmados localmente, es necesario enviarlos a la plataforma remota para que estén respaldados y disponibles para el equipo de desarrollo.

Enviar cambios (Push): Sube los commits locales a la rama principal (usualmente llamada 'main') en GitHub.

```
git push -u origin main
```

Descargar cambios (Pull): Trae y fusiona las últimas modificaciones del repositorio remoto a tu entorno de desarrollo local, asegurando la sincronización entre colaboradores.

```
git pull origin main
```

7. Buenas Prácticas Fundamentales

- **Hacer commits frecuentes:** Es preferible realizar múltiples commits pequeños y enfocados en una sola tarea que un único commit masivo con cambios heterogéneos.
- **Escribir mensajes claros:** Los mensajes deben ser descriptivos e imperativos (por ejemplo, "Corregir error en autenticacion" en lugar de "Cambios varios").
- **Revisar el estado constantemente:** Utiliza el comando `git status` para verificar en qué estado se encuentran tus archivos antes de agregarlos o confirmarlos.